AutoboxingDemo3.java

```java
void main()
{
     byte b = 12;
     Byte b1 = Byte.valueOf(b);
     IO.println(b1);

     short s = 15;
     Short s1 = Short.valueOf(s);
   IO.println(s1);

   int i = 90;
     Integer i1 = Integer.valueOf(i);
     IO.println(i1);


   long l = 12;
     Long l1 = Long.valueOf(l);
     IO.println(l1);

     float f = 15.4F;
     Float f1 = Float.valueOf(f);
   IO.println(f1);

   double d = 90.67;
     Double d1 = Double.valueOf(d);
     IO.println(d1);


    char c = 'A';
      Character c1 = Character.valueOf(c);
      IO.println(c1);


    boolean x = true;
      Boolean x1 = Boolean.valueOf(x);
    IO.println(x1);
}
```

ALL THE 8 WRAPPER CLASSES ARE HAVING valueOf() METHOD TO CONVERT THE PRIMITIVE INTO WRAPPER OBJECT

Autoboxing4.java

```java
void main()
{
     byte b = 12;
     Byte b1 = b;
     IO.println(b1);

     char c = 'A';
     Character c1 = c;
     IO.println(c1);

     boolean x = true;
     Boolean x1 = x;
     IO.println(x1);
}
```

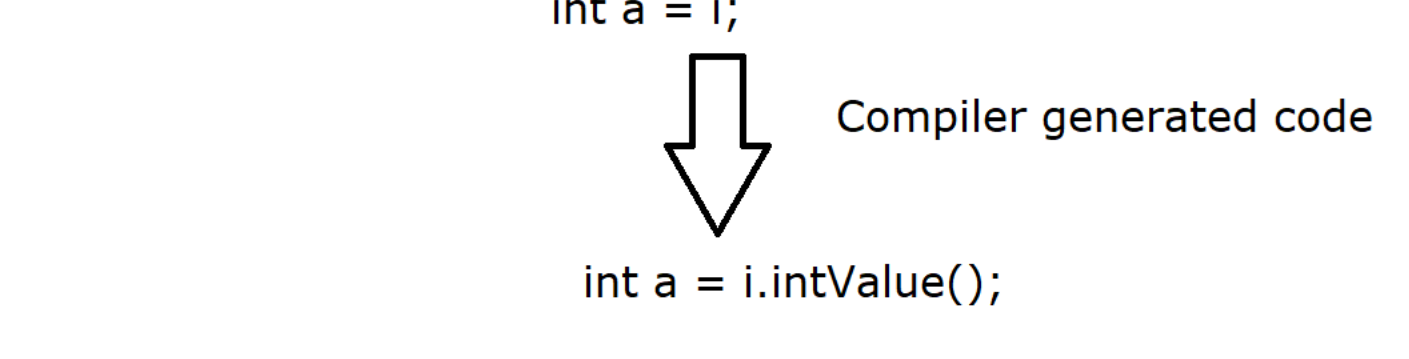
Due to Autoboxing concept we can directly assign primitive to wrapper from JDK 1.5V

Unboxing :

* It is the reverse process of Autoboxing.

* It is a technique though which we can convert **wrapper object back to primitive type.**

Wrapper Object		Primitive type
Byte	-	byte
Short	-	short
Integer	-	int
Long	-	long
Float	-	float
Double	-	double
Chracter	-	char
Boolean	-	boolean



* Till JDK 1.4V, It was the developer responsibility to convert the Wrapper object into primitive by writing xxxValue() method.
  Example :
```java
     Byte b1 = 90;
     byte b = b1.byteValue();  //Upto JDK 1.4V
     IO.println(b);
```

* From JDK 1.5V onwards we can directly assign the Wrapper object into corresponding primitive type due to the support of unboxing.
  Example :

```java
     Byte x = 100;
     byte y = x;     //JDK 1.5 [Unboxing]
```

```java
Integer i = 90;
int a = i;
```
⇩ Compiler generated code
```java
int a = i.intValue();
```

Method :
----
public int intValue() :
----
* It is a predefined non static method of Integer class though which we can convert the Integer Wrapper object back to int type (primitive type)
  Note : All these 8 warpper classes are having this method (xxxValue()) to convert wrapper object back to primitive

Unboxing.java

```java
void main()
{
     Byte b1 = 90;
     byte b = b1.byteValue();  //Upto JDK 1.4V
     IO.println(b);

     Short s1 = 100;
     short s = s1.shortValue();
     IO.println(s);

   Character c1 = 'A';
   char c = c1.charValue();
   IO.println(c);

   Boolean x = false;
   boolean y = x.booleanValue();
   IO.println(y);
}
```

ALL THESE 8 WRAPPER CLASSES ARE HAVING xxxValue() method to convert Wrapper into Primitive

Unboxing2.java

```java
void main()
{
     Byte b1 = 90;
     byte b = b1;
     IO.println(b);

     Short s1 = 100;
     short s = s1;
     IO.println(s);


   Integer i1 = 56;
     int i = i1.intValue();
   IO.println(i);

   Character c1 = 'A';
   char c = c1;
   IO.println(c);

   Boolean x = false;
   boolean y = x;
   IO.println(y);
}
```

Note : **Unlike primitive type, automatic type conversion is not possible with wrapper classes because these are objects so, the following statements are invalid.**

```java
Long x = 12;  //Invalid
Float f = 15;  //Invalid
Double d = 12; //Invalid
```

Unboxing3.java

```java
void main()
{
     /*
     Long x = 12;  //Invalid
   Float f = 15;  //Invalid
   Double d = 12; //Invalid
   */

     Long x = 12L;
     Float f = 12F;
     Double d = 12D;
     Double d1 = 12.90;
     IO.println(x);
     IO.println(f);
     IO.println(d);
     IO.println(d1);
}
```

Method Overloading Resolution : [Advanced Method Overloading]

In order to learn Method Overloading Resolution, we must have 3 knowledge of basic concepts

a) Var args in Java
b) Widening (implicit type conversion) in Java
c) Autoboxing in Java

In case of ambiguity where compiler can select more than one method then compiler will provide the priority to select a method by using following rules:

1) Exact match Rule :
----
* Based on the provided value (argument), Java compiler will search the method based on the exact match rule that means it will search a method with parameter with appropriate type.
  Example :
  Case 1 :   accept(9);   = javac will serach int type
  Case 2 :   accept(9L);   = javac will serach long type
  Case 3 :   accept('A'); = javac will serach char type

2) Most specific type OR Nearest type rule :
----
* If exact match is not found then javac will provide the priority based on the specific type OR nearest type.
  Example :
  double > float [float is the most specific type]
  float > long [long is the most specific type]
  long > int
  int > char
  int > short   [There is no specific type of relation in between **char & short**]
  short > byte

3) While working with Widening, Autoboxing and Var args, Java compiler will provide the priority based on
   WAV (Widening -> Autoboxing -> Var args)

//Program :

MOLR1.java

```java
void main()
{
     accept(6);
}

public void accept(double d)
{
     IO.println("double");
}
public void accept(float d)
{
     IO.println("float");
}
```

Note: Here float will be executed because float is the most specific type.

MOLR2.java

```java
void main()
{
  accept(6);
}

public void accept(int d)
{
 IO.println("int");
}
public void accept(char d)
{
 IO.println("char");
}
```

Here 6 is int type so int will be executed using exact match Rule.

MOLR3.java

```java
void main()
{
 accept();
}

public void accept(int ...d)
{
 IO.println("int");
}
public void accept(char ...d)
{
 IO.println("char");
}
```

char will be executed becoz char is more specific type.

MOLR4.java

```java
void main()
{
 accept();
}

public void accept(short ...d)
{
  IO.println("short");
}
public void accept(char ...d)
{
  IO.println("char");
}
```

Here we will get compilation error because there is no relation between char and short based on the specific type rule.

MOLR5.java

```java
void main()
{
  accept();
}

public void accept(short ...d)
{
  IO.println("short");
}
public void accept(byte ...d)
{
  IO.println("byte");
}
```

Here byte will be executed because byte is the specific type.

MOLR6.java

```java
void main()
{
 accept();
}

public void accept(double ...d)
{
 IO.println("double");
}
public void accept(long ...d)
{
 IO.println("long");
}
```

Here long will be executed because long is the most specific type.

MOLR7.java

```java
void main()
{
 accept(9);
}

public void accept(int d)
{
 IO.println("int");
}
public void accept(long s)
{
 IO.println("long");
}
```

Note : Here int will be executed because int is the exact match

---

MOLR8.java

```java
void main()
{
 accept(9);
}

public void accept(long s)
{
 IO.println("Widening");
}
public void accept(Integer i)
{
 IO.println("Autoboxing");
}
```

Here widening is having more priority

MOLR9.java

```java
void main()
{
 accept(9);
}

public void accept(int ...s)
{
     IO.println("Var args");
}
public void accept(Integer i)
{
   IO.println("Autoboxing");
}
```

Here Autoboxing will be executed.

MORL10.java

```java
void main()
{
 accept(9);
}

public void accept(int ...s)
{
     IO.println("Var args");
}
public void accept(int i)
{
   IO.println("int");
}
```

Here int will be executed

MOR11.java

```java
void main()
{
 accept(9);
}

public void accept(int ...s)
{
     IO.println("Var args");
}
public void accept(int []i)
{
   IO.println("int array");
}
```

Here we will get compilation error Internally var args is an array

---

MORL12.java

```java
void main()
{
     accept(9,2);
}

public void accept(int ...s)
{
     IO.println("Var args");
}
public void accept(long x , long y)
{
   IO.println("Widening");
}
```

Here Widening will be executed.

MOLR13.java

```java
void main()
{
     accept(1,9L); //int-long
     accept(1L,9); //long - int
     //accept(1,2); //CE
}

public void accept(int x, long y)
{
     IO.println("int - long");
}
public void accept(long x , int y)
{
   IO.println("long - int");
}
```

MOL14.java

```java
void main()
{
     accept(9);
}

public void accept(Double y)
{
     IO.println("Double");
}
public void accept(Long x)
{
   IO.println("Long ");
}
```

Note : Compilation error
====================================